

Data Structures

Ezaz Ahmed

C223009 (3AM)

Topic: Introduction and Overview on Data Structures

Data: Data means value or set of values.

Example:

34

09/06/1974

☐ A data item refers to a single unit of values. Data items that are divided into sub items are called group items those that are not are called elementary items.

* Collections of data are frequently organized into a hierarchy of fields (attributes), records (entity) and files (entity set).

* An entity is something that has certain attributes which may be assigned values.

Entity : EMPLOYEE

Attributes : NAME AGE SEX DESIGNATION

Values : Mr. X 30 M Director

Entities with similar attributes constitute an entity set.

* Each record (entity) in a file (entity set) may contain many field (attribute) items. But the value in a certain field may uniquely determine the record in the file. Such a field K is called a primary Key.

* The term information is used for data with its attributes.

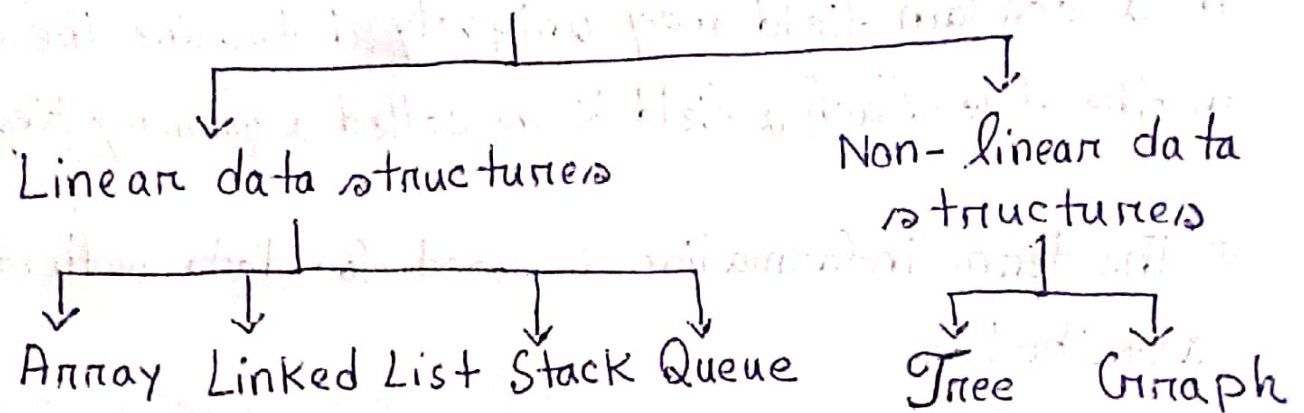
Data type:

Built in data type: int, float, char, double etc.

Abstract data type: A kind of a data type whose behaviour is defined with the help of some attributes and some functions.

Data Structure: The logical or mathematical model of a particular organization of data is called a data structure.

Data Structures



Floor Functions:

$\lfloor x \rfloor$ is called the floor of x , denotes the greater integer that does not exceed x .

Ceil Functions:

$\lceil x \rceil$ is called the ceiling of x , denotes the least integer that is not less than x .

Example:

$$\boxed{3.14} \quad \lfloor 3.14 \rfloor = 3 ; \lceil 3.14 \rceil = 4 ; \lfloor -8.5 \rfloor = -9 ;$$

$$\lceil -8.5 \rceil = -8$$

Remainder Function:

*Let k be any integer and let M be a positive integer. Then $k \pmod{M}$ will ~~denote~~ denote the integer remainder when k is divided by M .

$$25 \pmod{7} = 4$$

Congruence:

$a \equiv b \pmod{M}$ if and only if M divides $a-b$

$$22 \equiv 2 \pmod{5}$$

Algorithmic Notation

An algorithm is a finite step-by-step list of well-defined instructions for solving a particular problem.

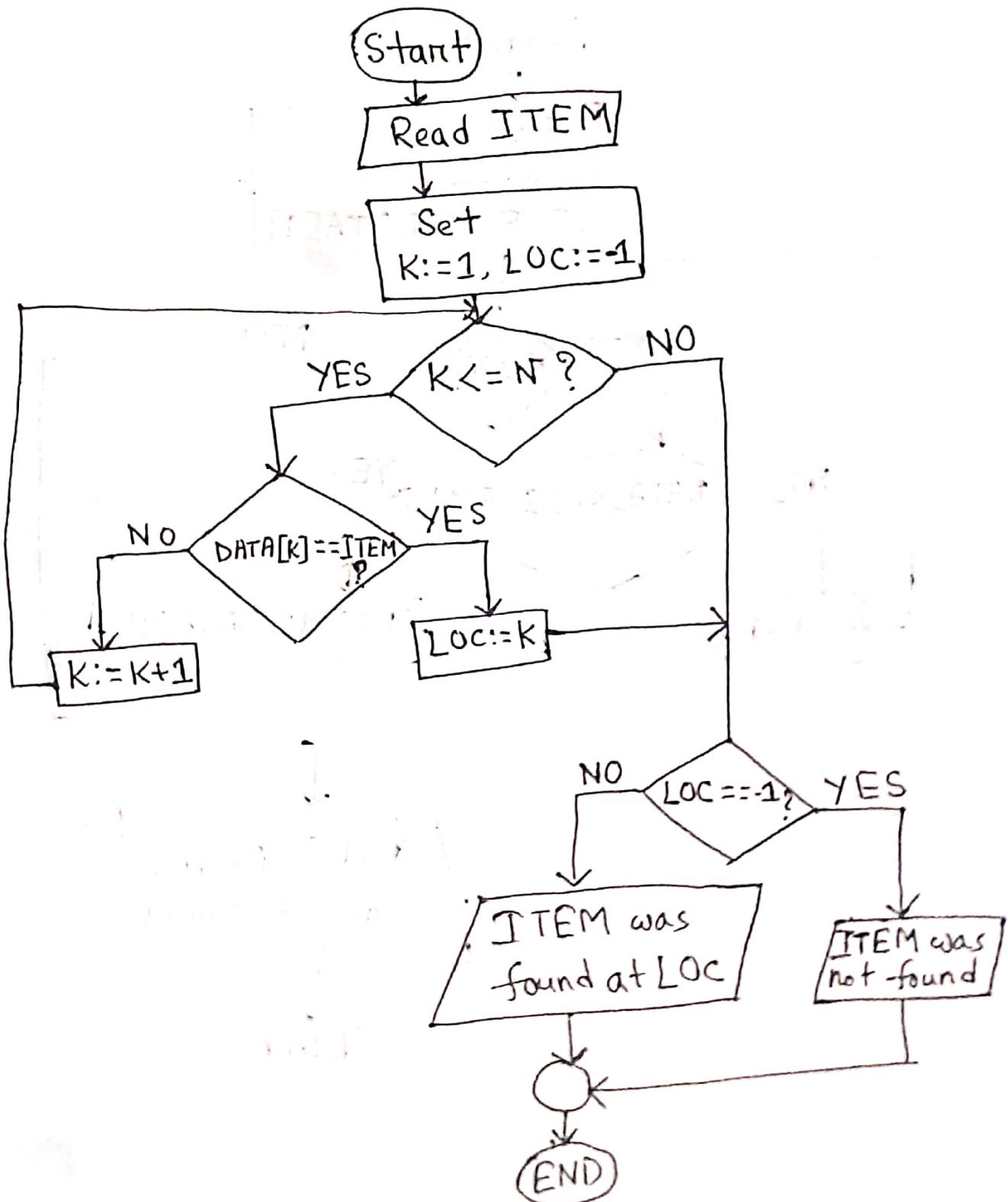
* Dots-equal notation $:=$ is used as assignment operation. Some texts use backward arrow $\leftarrow$ or the equal sign $=$ for this operation.

* For input Read: variables name is used.

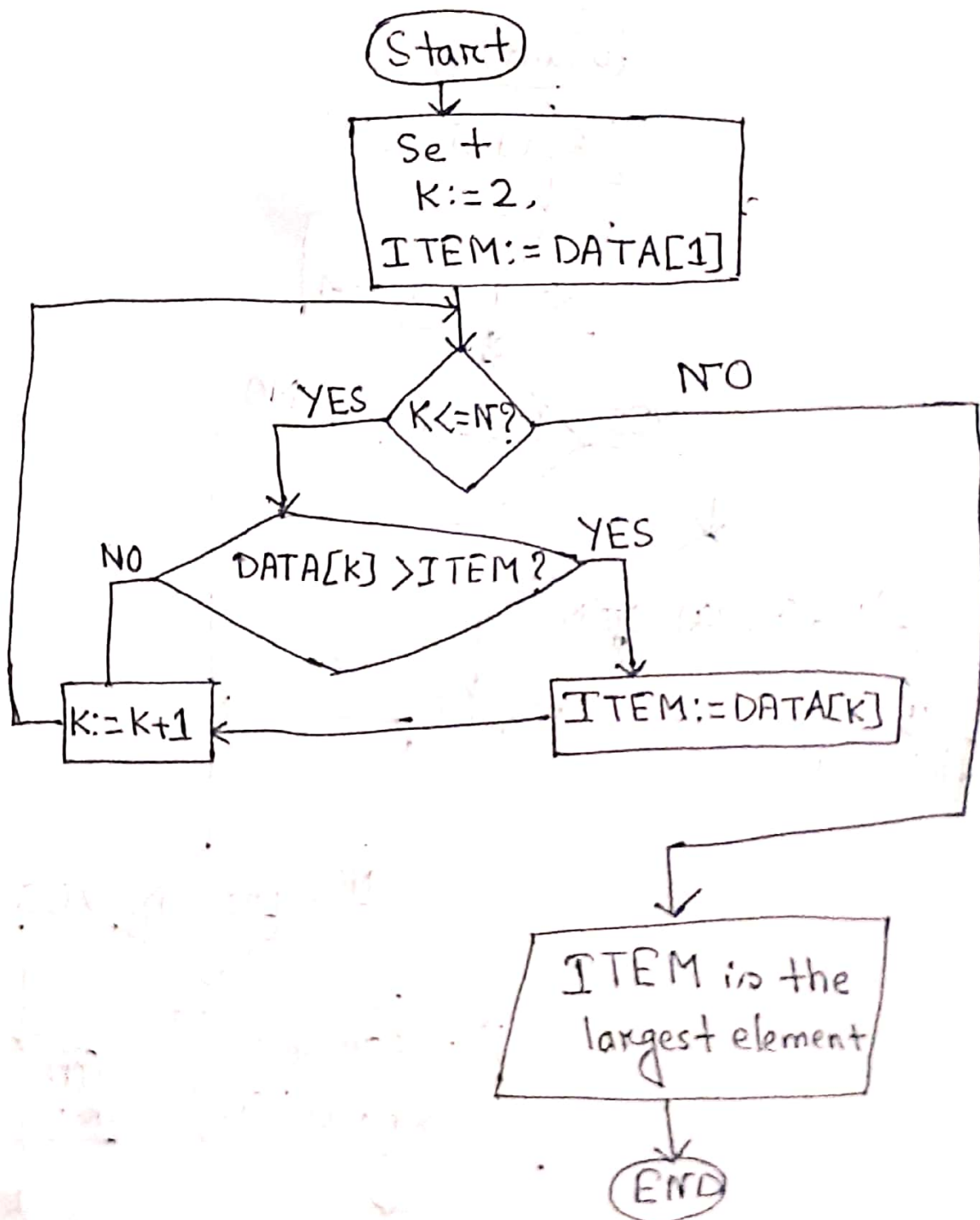
For writing Write: Messages and/or variables names.

Flowchart

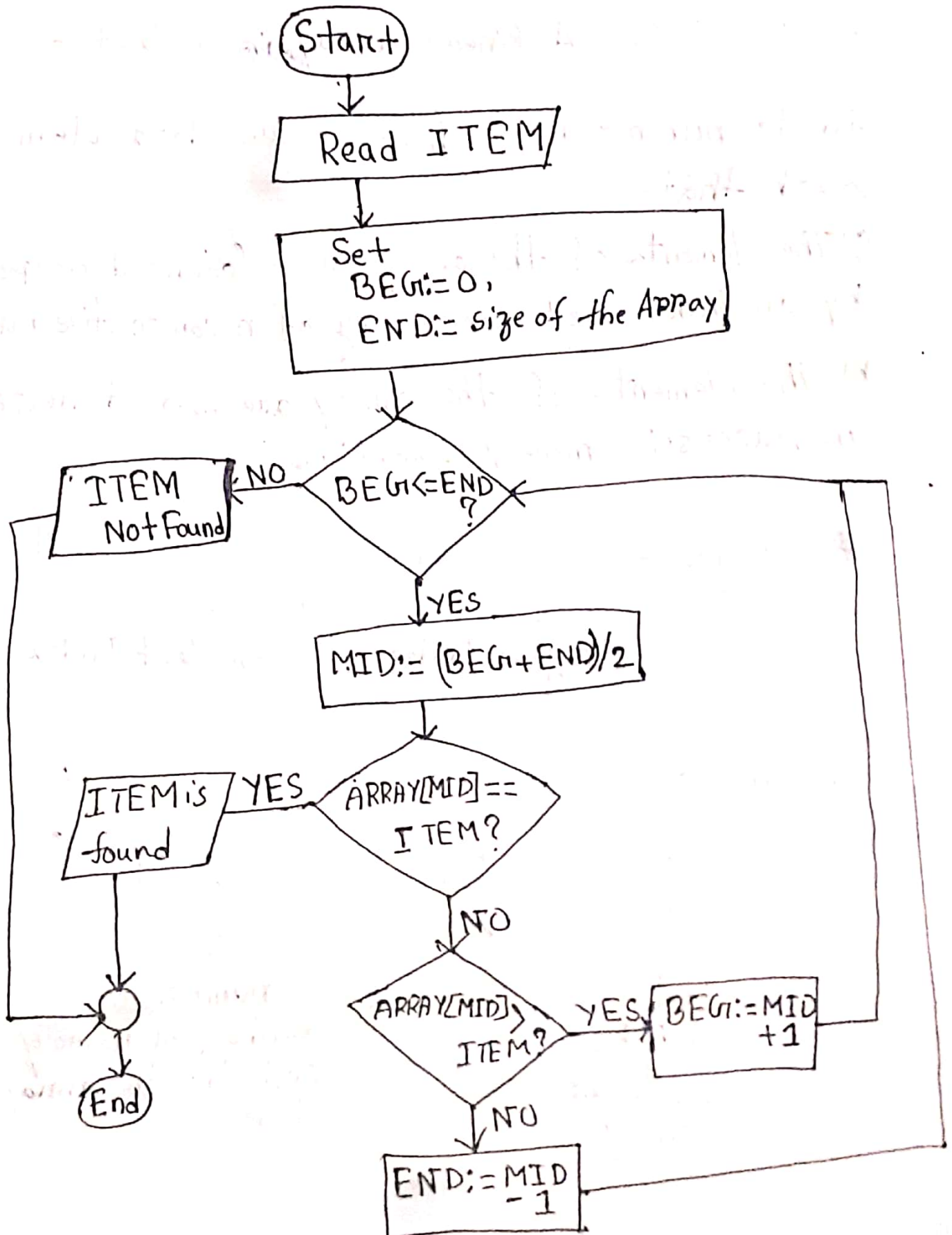
Linear Search:



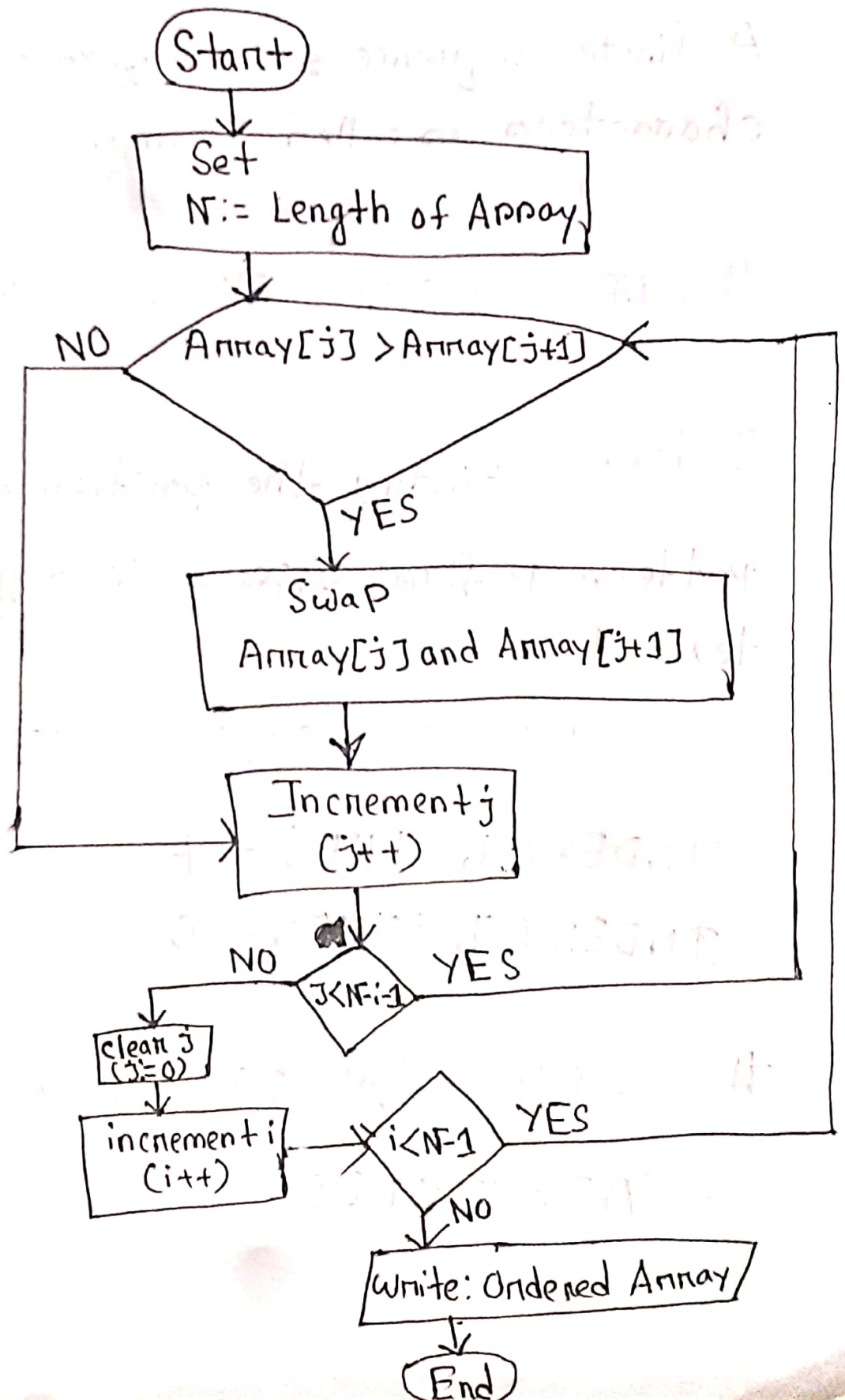
Largest Element in Array:



Binary Search:



Bubble Sort:



Ezaz Ahmed
c223009

Topic - Arrays =

Linear Array: A linear array is a list of a finite number n of homogeneous data elements such that-

- The elements of the array are referenced respectively by an index set consisting of n consecutive numbers.
- The elements of the array are stored respectively in successive memory locations.

$$* \text{Length} = \text{Upper Bound} - \text{Lower Bound} + 1$$

$\downarrow$
Largest Index

$\downarrow$
Smallest Index

Formula:-

$$\text{LOC}(\text{LA}[K]) = \text{Base}(\text{LA}) + w(K - \text{LB})$$

$\downarrow$
Let:
LA be
a linear
array

$\downarrow$
number of
words per memory
cell for the array
LA

Example:

Base(AUTO) = 200, $w = 4$ words per memory cell
for AUTO

Now,

$$\text{LOC}(\text{Auto}[1965]) = \text{Base}(\text{AUTO}) + w(1965 - \text{lower bound})$$

$$= 200 + 4(1965 - 1932)$$

$$= 332$$

প্রক্সে
চূড়ান্ত
Lower bound

Traversing Linear Arrays:

① Set $K := LB$

② Repeat Steps 3 and 4 while $K \leq UB$

Apply PROCESS to $LA[K]$

→ Array Name

③ Set $K := K + 1$.

[End of step 2 Loop]

④ Exit.

Inserting into a Linear Array:

INSERT (LA, N, K, ITEM)

↓ ↓ ↓
Array elements position
Name

Algorithm:

- ① Set $J := N$
- ② Repeat steps 3 and 4 while $J \geq K$ or, $J >= K$
- ③ Set $LA[J+1] := LA[J]$
- ④ Set $J := J - 1$
- ⑤ Set $LA[K] := ITEM$
- ⑥ Set $N := N + 1$
- ⑦ Exit.

Deleting :

- ① Set $ITEM := LA[K]$
- ② Repeat for $J = K$ to $N-1$
Set $LA[J] := LA[J+1]$
- ③ Set $N := N-1$
- ④ Exit.

Bubble Sort: (N elements)

- ① Repeat steps 2 and 3 for $K = 1$ to $N-1$
- ② Set $PTR := 1$
- ③ Repeat while $PTR \leq N-K$
 - a) If $DATA[PTR] > DATA[PTR+1]$ then
Interchange $DATA[PTR]$ and $DATA[PTR+1]$
 - b) Set $PTR := PTR + 1$
- ④ Exit.

Step-1

No swap
4 9 5 1 0

↓

swap
4 9 5 1 0

↓

swap
4 5 9 1 0

↓

swap
4 5 1 9 0

↓

4 5 1 0 9

Step-2

No swap
4 5 1 0 9

↓

swap
4 5 1 0 9

↓

swap
4 1 5 0 9

↓

4 1 0 5 9

Step-3

No swap
4 1 0 5 9

↓

swap
1 4 0 5 9

↓

1 0 4 5 9

Step-4

~~1~~ 0 4 5 9

↓

0 1 4 5 9

(Bubble Sorting)

Linear Search Algorithm:

- ① Set $K := 1$ and $LOC := 0$
- ② Repeat Steps 3 and 4 while $LOC = 0$ and $K \leq N$
- ③ If $ITEM = DATA[K]$, then: Set $LOC := K$
- ④ Set $K := K + 1$
- ⑤ If $LOC = 0$ then:
Write: ITEM is not found.
Else:
Write: ITEM is found in index LOC.
- ⑥ Exit.

Binary Search Algorithm:

- ① Set $BEG := LB$, $END := UB$ and $MID := INT$
 $MID := INT((BEG + END) / 2)$
- ② Repeat Steps 3 and 4 while $BEG \leq END$
and $DATA[MID] \neq ITEM$

③ If $ITEM < DATA[MID]$ then:

Set $END := MID - 1$

Else:

Set $BEG := MID + 1$

④ Set $MID := INT((BEG + END) / 2)$

⑤ If $DATA[MID] = ITEM$, then:

Set $LOC := MID$

Else:

Set $LOC := Null$

⑥ Exit.

Complexity of Algorithms

The complexity of an algorithm M is the function $f(n)$ which gives the running time and/or ~~storage~~ storage space requirement of the algorithm in terms of the size n of the input data.

a) Worst Case: the combination of input data for which the complexity $f(n)$ is maximum.

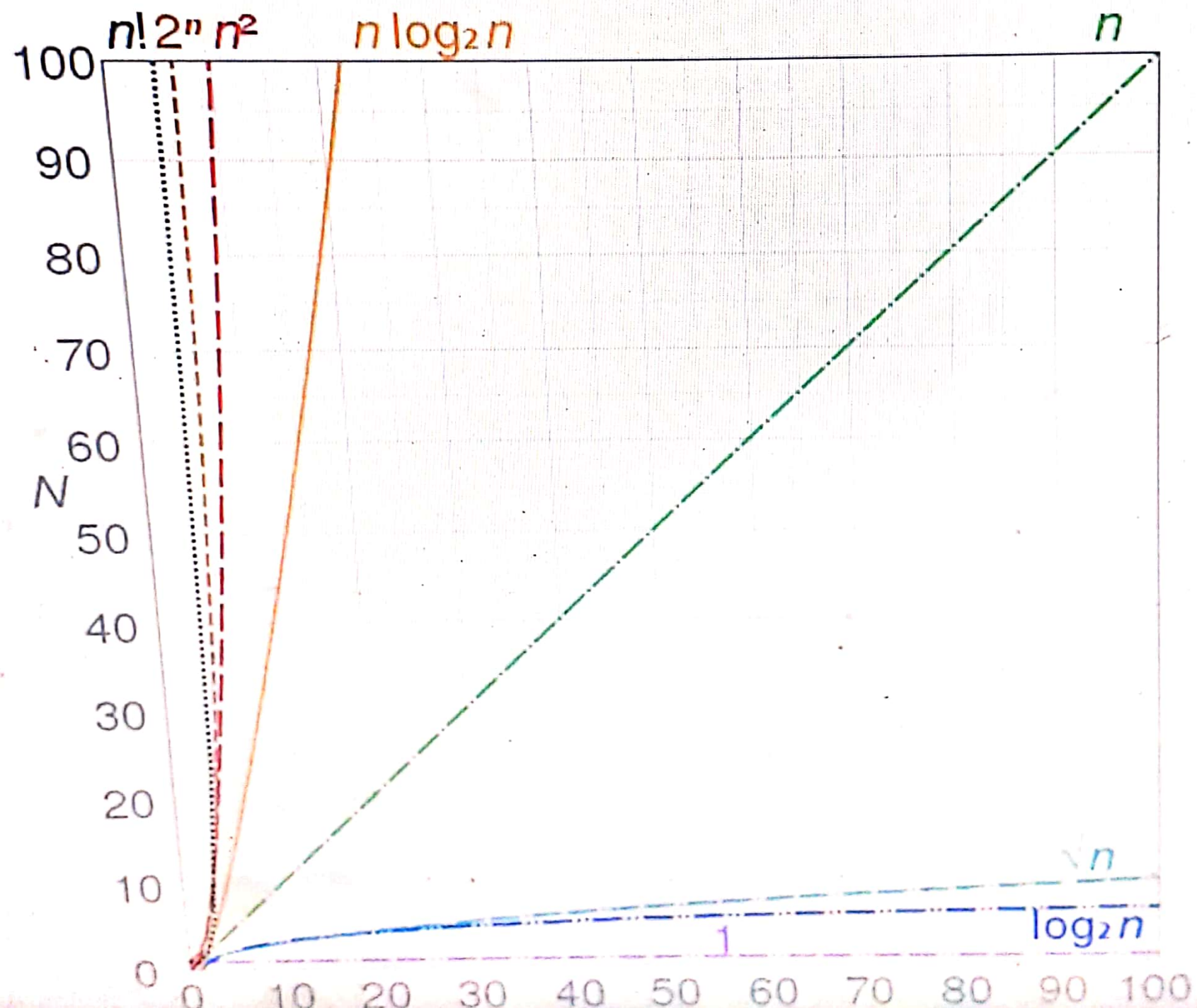
b) Average Case: the combination of an input data for which the complexity $f(n)$ is not maximum and not minimum but has an expected value.

c) Best Case: the combination of input data for which the complexity $f(n)$ is minimum.

⊞ Asymptotic Notations: Mathematical tools to represent time complexity of algorithms for asymptotic analysis.

① O Notation ② Ω Notation ③ Θ Notation

Rate of Growth

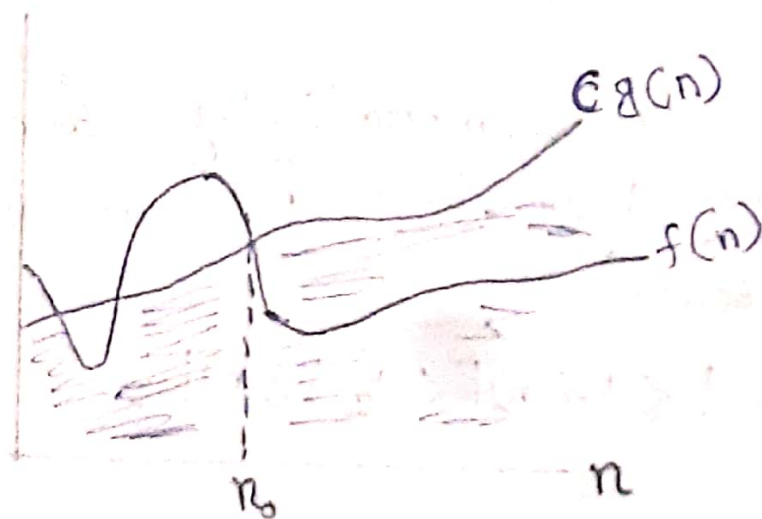


Big O Notation: The running time grows at most this much, but it could grow more slowly. In this occasions we use "Big-O" notation.

Suppos $f(n)$ and $g(n)$ are functions defined on the positive integers with the property that $f(n)$ is bounded by some multiple of $g(n)$ for almost all n . Suppose there exist a positive integer n_0 and a positive integer C so for all $n > n_0$,

$$|f(n)| \leq C |g(n)|$$

$$f(n) = O(g(n)) \quad \text{" } f(n) \text{ is the order of } g(n) \text{"}$$

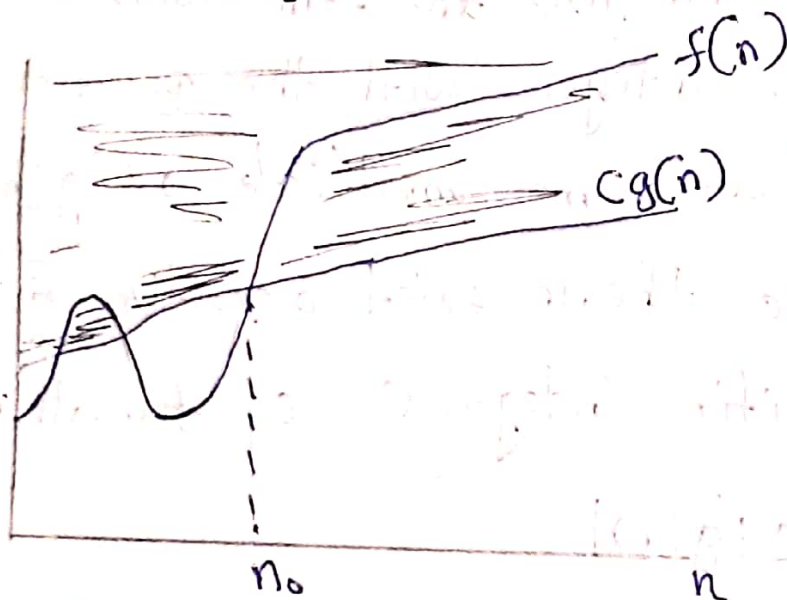


Ω notation

=

Sometimes we want to say that an algorithm takes at least a certain amount of time, without providing an upper bound then we use Ω notation.

$$f(n) \gg c |g(n)|$$

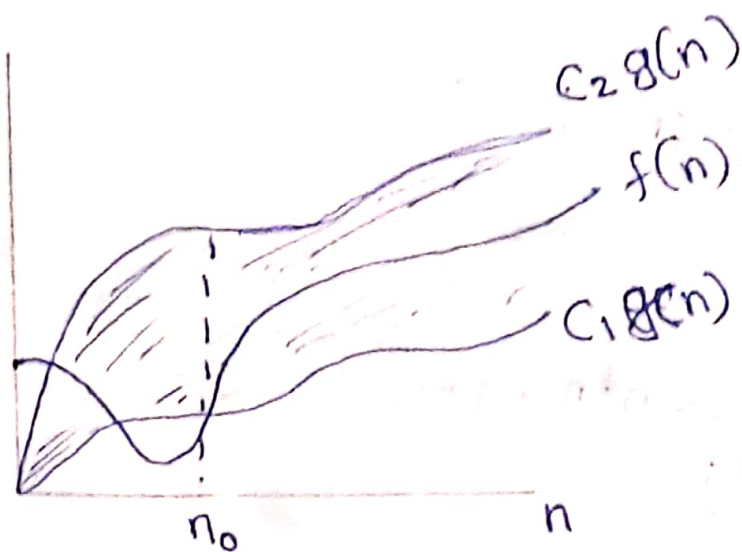


Θ notation

=

We use it to asymptotically bound the growth of a running time to within constant factors above and below.

$$C_1 |g(n)| \leq |f(n)| \leq C_2 |g(n)|$$



Example:

Writing a function in Big-O =

- # Only consider the highest-order term.
- # Ignore any coefficients or constant factors.

Example:

```
sum = 0
for (i = 0; i < n; i++)
  for (j = 0; j < i * i; j++)
    for (k = 0; k < j; k++)
      sum++;
```

Answer: $O(n^5)$

sum = 0;

for (i = 0; i < n; i++)

sum++;

val = 1;

for (j = 0; j < n * n; j++)

val = val * j

Time (n + n²)

Ans: O(n²)

Space Complexity

=

Space complexity is the total amount of memory space used by an program including the space of input values for execution.

Space Complexity = Auxiliary space + Space use by
input values

If a program takes up a lot of memory space, the compiler will not let you run it.

Example:

```
int sum (int a, int b, int c) {  
    int r = a + b + c;  
    return r;  
}
```

Ans: $O(1)$

Complexity of Linear Search Algorithm
 $= O(n)$

Worst Case: When ITEM is the last element in the array DATA or is not there at all. $f(n) = n = O(n)$

Average Case:

$$\begin{aligned} f(n) &= 1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + 3 \cdot \frac{1}{n} + \dots + n \cdot \frac{1}{n} \\ &= (1 + 2 + 3 + \dots + n) \cdot \frac{1}{n} \\ &= \frac{n(n+1)}{2} \cdot \frac{1}{n} = \frac{n+1}{2} = \frac{n}{2} + \frac{1}{2} = O(n) \end{aligned}$$

Complexity of Binary Search Algorithm

$$2^{f(n)} > n = \log_2 n + 1$$

That is the running time for the worst case is approximately equal to $\log_2 n$. One can also show that the running time for the average case is approximately equal to the running time for the ~~worst~~ worst case.

Complexity of Bubble Sort Algorithm

$$f(n) = (n-1) + (n-2) + \dots + 2 + 1 = n*(n-1)/2$$

$$= \frac{\tilde{n} - n}{2}$$

$$= O(\tilde{n})$$

Topic: Multidimensional Arrays

=

* A multi-dimensional array is an array of arrays.

Representation of Two-Dimensional
Array in Memory

$(M \times N)$

$$LOC(A[J,K]) = Base(A) + w [M(K-1) + (J-1)]$$

[Column-Major]

$$LOC(A[J,K]) = Base(A) + w [N(J-1) + (K-1)]$$

[Row-Major]

Example:

(25×4)

Base(MARKS) = 100, $w=4$ [Row Major]

$$\begin{aligned} LOC(MARKS[10,2]) &= 100 + 4 [4(10-1) + (2-1)] \\ &= 100 + 4 \times 37 = 248 \end{aligned}$$

MAZE (2:8, -4:1, 6:10)

Lengths are,

$$L_1 = 8 - 2 + 1 = 7, L_2 = 1 - (-4) + 1 = 6, L_3 = 10 - 6 + 1 = 5$$

MAZE contains $L_1 \cdot L_2 \cdot L_3 = 7 \cdot 6 \cdot 5 = 210$ elements.

If stored MAZE in row-major order and suppose $\text{Base}(\text{MAZE}) = 200$, $w = 4$, The address of an element of MAZE, $\text{MAZE}[5, -1, 8] = ?$

Now;

$$E_1 = 5 - 2 = 3, E_2 = -1 - (-4) = 3, E_3 = 8 - 6 = 2$$

Now,

$$E_1 L_2 = 3 \cdot 6 = 18$$

$$E_1 L_2 + E_2 = 18 + 3 = 21$$

$$(E_1 L_2 + E_2) L_3 = 21 \cdot 5 = 105$$

$$(E_1 L_2 + E_2) L_3 + E_3 = 105 + 2 = 107$$

Therefore,

$$\text{LOC}(\text{MAZE}[5, -1, 8]) = 200 + 4 \cdot (107) = 628$$

Note: If stored in column Major, the formula is:-

$$LOC(A[i, j, k]) = Base(A) + w[(E_3 L_2 + E_2) L_1 + E_1]$$

Matrix Multiplication:

Let A be an $M \times P$ matrix array and B ^{be} ~~is~~ a $P \times N$ matrix array.

- ① Repeat Steps 2 to 4 for $I = 1$ to M :
- ② Repeat Steps 3 and 4 for $J = 1$ to N :
- ③ Set $C[I, J] := 0$
- ④ Repeat for $K = 1$ to P :
 $C[I, J] := C[I, J] + A[I, K] * B[K, J]$
- ⑤ Exit.

Sparse Matrices:

Matrices with a relatively high proportion of zero entries is called sparse matrices.

If the number of zeros in a matrix exceeds $(n*m)/2$ where n and m are dimension of the matrix then the matrix is called sparse matrix.

Topic: String



A finite sequence S of zero or more characters is called String.

SUBSTRING ('THE END', 4, 4) = 'END'
 ↓
 Space

Indexing: Finding the position where a string pattern P first appears in a giving string text T .

T = 'His FATHER IS THE PROFESSOR'

$$\text{INDEX}(T, \text{'THE'}) = 7$$
$$\text{INDEX}(T, \text{'THEN'}) = 0$$

```
# INSERT ('ABCDEF', 3, 'XYZ')
```

= 'ABXYZCDEF'

DELETE('ABCDEFGH', 4, 2) = 'ABCFGH'

REPLACE('XABYABZ', 'AB', 'C') = 'XCYABZ'

Replacement: (Every occurrence)

$T = XAY$; $P = A$, $Q = AB$

Now after replacing:

$T = XABY$ at the end of the first execution

$T = XAB^2Y$ " " " " second "

.....

$T = XAB^nY$ at the end of the n^{th} ~~execu-~~ execution of the loop.

Pattern Matching Algorithm:

P and T are strings with length R and S

① Set $K := 1$ and $Max := S - R + 1$

② Repeat steps 3 to 5 while $K \leq Max$:

③ Repeat for $L = 1$ to R :

If $P[L] \neq T[K+L-1]$, then: Go to step 5.
[END of inner loop]

④ [Success] Set $INDEX := K$ and Exit.

⑤ Set $K := K + 1$.

[End of step 2 outer loop]

⑥ [Failure] Set $INDEX = 0$.

⑦ Exit.

Example:

$P = abba$, $T = ababbaabaaab$

Now,

abba _ _ _ _ _	step 1	ABA#	mismatch 4th letter
- abaa _ _ _ _ _	step 2	- #...	mismatch 1st letter
_ _ abaa _ _ _ _ _	step 3	_ _ AB#.	mismatch 3rd letter
_ _ _ abaa _ _ _ _ _	step 4	_ _ _ #...	mismatch 1st letter
_ _ _ _ abaa _ _ _ _ _	step 5	_ _ _ _ #...	mismatch 1st letter
_ _ _ _ _ abaa _ _ _ _ _	step 6	_ _ _ _ _ A#..	mismatch 2nd letter
_ _ _ _ _ _ abaa _ _	step 7	_ _ _ _ _ _ ABAA	success

Total comparisons, $C = (4+1+3+1+1+2) = 12$

$INDEX(P, T) = 7$

Pattern Matching Algorithm

```
for (j = 0; j ≤ n - m; j++) {  
    for (i = 0; i < m && x[i] == y[i+j]; i++);  
    if (i == m) return j;  
}
```

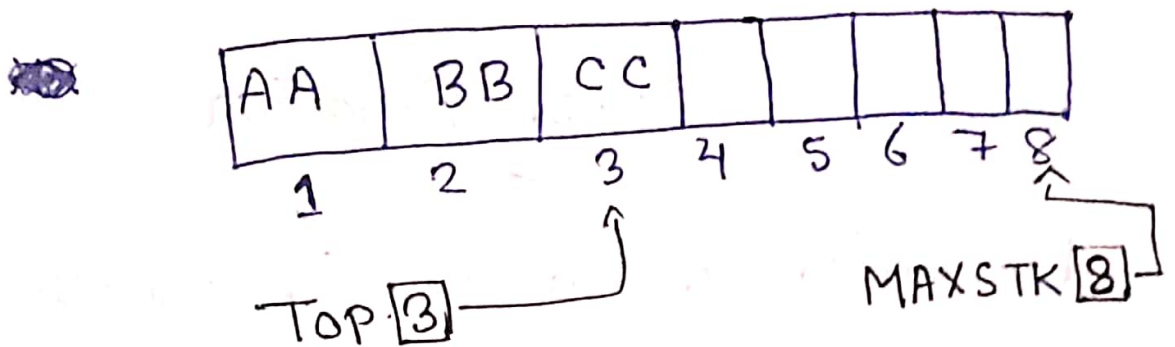
Feature of this easy but slow $O(nm)$ algorithm:

- * No preprocessing phase.
- * Only constant extra space needed.
- * Always shifts the window by exactly 1 position to the right.
- * Comparisons can be done in any order.
- * mn expected text characters comparisons.

Topic - Stacks =

A stack is a list of elements in which an element may be inserted or deleted only at one end, called the top of the stack.

Stacks are called last-in-first-out (LIFO) lists.



Algorithm PUSH (STACK, TOP, MAXSTK, ITEM):

① [STACK already filled]? If $TOP = MAXSTK$ then: Print: OVERFLOW, and Return.

② Set $TOP := TOP + 1$

③ Set $STACK[TOP] := ITEM$

④ Return.

Algorithm POP(STACK, TOP ITEM):

- ① If $TOP = 0$ then: Print: UNDERFLOW and Return.
- ② Set $ITEM := STACK[TOP]$
- ③ Set $TOP := TOP - 1$
- ④ Return.

Polish Notation
=

Evaluation of a post-fix expression:

~~val~~ VALUE of an arithmetic expression P written in post-fix notation.

- ① Add a right parenthesis ")" at the end of P .
- ② Scan P from left to right repeat steps 3 and 4 for each element of P until the sentinel ")" is encountered.

③ If an operand is encountered, put it in the STACK.

④ If an operator θ is encountered, then:

ⓐ Remove the two top elements of STACK, where A is the top element and B is the next-to-top element.

ⓑ Evaluate $B \theta A$

ⓒ Place the result (b) back on STACK.

⑤ Set VALUE equal to the top element on STACK.

⑥ Exit.

Top = 4 ,

STACK: 227, 123, N, 583, —, —, —, —

* Call POP(STACK, ITEM A)

ITEM A = 583

STACK: 227, 123, N, —, —, —, —
Top = 3

* Call POP(STACK, ITEM B)

ITEM B = N

STACK: 227, 123, —, —, —, —, —
Top = 2

* Call POP(STACK, ITEM C)

ITEM C = 123

STACK: 227, —, —, —, —, —, —
Top = 1

* Call PUSH(STACK, ITEM A + 140)

STACK: 227, 723, —, —, —, —, —
Top = 2

* Call PUSH(STACK, 190)

STACK: 227, 723, 190, —, —, —, —
Top = 3

* Call PUSH(STACK, ITEM B + ITEM C)

STACK: 227, 723, 190, N + 123, —, —, —
Top = 4

* Call PUSH(STACK, ITEM A + ITEM B)

STACK: 227, 723, 190, N + 123, 583 + N, —, —
Top = 5

Repeat while $Top \neq 0$

Call POP(STACK, ITEM)

Write: ITEM

STACK: -, -, -, -, -, -, -, - (TOP = 0)

Output:

583+N

N+123

190

723

227

Transforming Infix to Postfix:

POLISH(Q, P)

$\rightarrow$ Postfix expression

$\rightarrow$ Infix expression

① ~~PUSH~~ Push "(" onto STACK, and add ")" to the end of Q.

② Scan Q from left to right and repeat Steps 3 to 6 for each element of Q until the STACK is empty.

③ If an operand is encountered, add it to P.

④ If a left parenthesis is encountered, push it onto STACK.

⑤ If an operator $\otimes$ is encountered, then:

 a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) which has the same precedence as or higher precedence than $\otimes$.

 b) Add $\otimes$ to STACK.

⑥ If a right parenthesis is encountered then:

① Repeatedly pop from STACK and add to Peach operator until a left parenthesis is encountered.

② Remove the left parenthesis.

⑦ Exit.

Example:

$$A + (B * C - (D / E \uparrow F) * G) * H$$

We push "(" onto STACK and then we add ")" to the end of Q:

Q: A + (B * C - (D / E $\uparrow$ F .) * G) * H)
① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩ ⑪ ⑫ ⑬ ⑭ ⑮ ⑯ ⑰ ⑱ ⑲ ⑳

Symbol Scanned	STACK	Expression P
① A	(	A
② +	(+	A
③ (	(+(	A
④ B	(+(	AB
⑤ *	(+(*	AB
⑥ C	(+(*	ABC
⑦ -	(+(-	ABC*
⑧ (	(+(-(	ABC*
⑨ D	(+(-(	ABC*D
⑩ /	(+(-(	ABC*D
⑪ E	(+(-(	ABC*DE
⑫ ↑	(+(-(↑	ABC*DE
⑬ F	(+(-(↑	ABC*DEF
⑭)	(+(-	ABC*DEF↑/

⑮ *	(+ (- *	ABC * DEF ↑ /
⑯ G	(+ (- *	ABC * DEF ↑ / G
⑰)	(+	ABC * DEF ↑ / G * -
⑱ *	(+ *	ABC * DEF ↑ / G * -
⑲ H	(+ *	ABC * DEF ↑ / G * - H
⑳)		ABC * DEF ↑ / G * - H * +

Thanks Everyone
Assalamualikum